Member-only story

Automated Barman Backup + Pgbarman + Barman Incremental Backup + Barman Full Backup +

Automating PostgreSQL Backups with Barman

Sheikh Wasiu Al Hasib Follow 5 min read · Dec 2, 2025

Full Every Friday, Incremental Twice a Day

Topic: PostgreSQL · Barman · Backup Automation · Bash

Backing up a production PostgreSQL database is not just a best practice — it's a survival rule.
If you're using **Barman (Backup and Recovery Manager)**, you already have a powerful tool for managing full and incremental backups, WAL archiving, and point-in-time recovery (PITR).

In this post, I'll share a simple Bash script that automates:

- **Full backup** every **Friday at 00:00**
- **Incremental backup** every day at **08:00** and **16:00**
- Plus a quick call to `barman cron` for maintenance

The idea is to keep backups:

- **Regular enough** to reduce data-loss window
- **Lightweight enough** not to overload the system
- **Automated enough** that you don't depend on manual operations

1. Prerequisites

Before using the script, you should already have:

- A working **PostgreSQL** primary server
- **Barman** installed on a separate backup server
- A configured Barman server entry, e.g. `pgcomp` in `/etc/barman.conf` or `/etc/barman.d/pgcomp.conf`
- Passwordless or secure access from the Barman host to PostgreSQL (for `barman backup`)

If `barman backup pgcomp` runs successfully from the command line, you're ready to automate.

2. The Backup Script

Here's the full script:

```
#!/bin/bash
# Author : Sheikh Wasiu Al Hasib
# Description: barman_backup - Full every Friday 00:00, Incremental every day at
SERVER="pgcomp"        # ← CHANGE TO YOUR REAL SERVER NAME
DAY=$(date +%u)        # 1 = Monday ... 5 = Friday ... 7 = Sunday
HOUR=$(date +%H)

# Friday at 00:00 > Force FULL backup
if [[ $DAY -eq 5 && $HOUR -eq 00 ]]; then
    echo "$(date '+%Y-%m-%d %H:%M:%S') - FULL backup (Friday midnight)"
    /usr/bin/barman backup --reuse-backup=off "$SERVER"
# Every day at 08:00 and 16:00 > Incremental
elif [[ $HOUR == 08 || $HOUR == 16 ]]; then
    echo "$(date '+%Y-%m-%d %H:%M:%S') - INCREMENTAL backup ($HOUR:00)"
    /usr/bin/barman backup "$SERVER"
else
    echo "$(date '+%Y-%m-%d %H:%M:%S') - No backup scheduled at this time"
    exit 0
fi
# Maintenance
/usr/bin/barman cron
```

> 🔁 You'll usually save this as something like:
> `/usr/local/bin/barman_backup.sh` and make it executable:

```
chmod +x /usr/local/bin/barman_backup.sh
```

3. Understanding the Logic

Let's walk through each part of the script and see what it's doing.

3.1 Server Name

```
SERVER="pgcomp"        # ← CHANGE TO YOUR REAL SERVER NAME
```

- `SERVER` must match the Barman server name defined in your Barman config.
- Example: if you have `/etc/barman.d/pgcomp.conf`, the server name is `pgcomp`.

3.2 Current Day and Hour

```
DAY=$(date +%u)        # 1 = Monday ... 5 = Friday ... 7 = Sunday
HOUR=$(date +%H)
```

- `DAY` uses `+%u` → numeric day of week:
- `1` = Monday
- `2` = Tuesday
- `5` = Friday
- `7` = Sunday
- `HOUR` uses `+%H` → **24-hour format** (`00` – `23`).

This lets the script make decisions based on **day + time**, even if you call it every hour via cron.

3.3 When to Run a Full Backup

```
# Friday at 00:00 > Force FULL backup
if [[ $DAY -eq 3 && $HOUR -eq 00 ]]; then
    echo "$(date '+%Y-%m-%d %H:%M:%S') - FULL backup (Friday midnight)"
    /usr/bin/barman backup --reuse-backup=off "$SERVER"
```

Conceptually, this block says:

- If **day is Friday**
- And **hour is 00 (midnight)**
- Then run a **FULL backup**.

The important part is:

```
/usr/bin/barman backup --reuse-backup=off "$SERVER"
```

- `barman backup SERVER` → starts a backup for that server.
- `--reuse-backup=off` → **forces a fresh full backup**, not incremental reuse of old data.

So, **once a week** you get a clean, full backup baseline.

> ⚠️ Note: As written, `DAY -eq 3` actually means **Wednesday** (not Friday).
> If you want Friday, you should use `DAY -eq 5`.

- `1` = Monday
- `2` = Tuesday
- `3` = Wednesday
- `4` = Thursday
- `5` = Friday

If your intention is **Friday**, update this line to:

```
if [[ $DAY -eq 5 && $HOUR -eq 00 ]]; then
```

3.4 When to Run Incremental Backups

```
# Every day at 08:00 and 16:00 > Incremental
elif [[ $HOUR == 08 || $HOUR == 16 ]]; then
    echo "$(date '+%Y-%m-%d %H:%M:%S') - INCREMENTAL backup ($HOUR:00)"
    /usr/bin/barman backup "$SERVER"
```

- For **every day**, when the hour is `08` or `16`, the script runs:

```
/usr/bin/barman backup "$SERVER"
```

Depending on your Barman configuration and PostgreSQL version, this will:

- Either perform a standard backup that **reuses** previous backup data (if `--reuse-backup` default is on), or
- Use **incremental** mode (e.g., with PostgreSQL 15+ and Barman's incremental support).

The idea is:

- You get **two restore points per day** above the weekly full.
- The backup volume is reduced because Barman will reuse unchanged data blocks where possible.

3.5 When No Backup is Scheduled

```
else
    echo "$(date '+%Y-%m-%d %H:%M:%S') - No backup scheduled at this time"
    exit 0
fi
```

If the script runs at any other time (for example, 03:00, 11:00, 23:00, etc.):

- It prints a message that no backup is scheduled.
- Exits with `0` (no error).

This is important because **cron** may run the script more often (for example, every hour), but we don't want to trigger a backup every time.

3.6 Barman Maintenance: `barman cron`

```
# Maintenance
/usr/bin/barman cron
```

This line runs Barman's scheduled maintenance tasks:

- WAL archive checks
- Backup catalog rotations

- Retention policy maintenance
- Other internal housekeeping

The official docs recommend running `barman cron` regularly (e.g., via system cron). Adding it at the end of the backup script is a nice way to ensure it runs after a backup cycle.

4. Scheduling the Script with Cron

The script itself only decides **what to do based on DAY + HOUR.**
You still need to schedule it with **crontab**.

A common pattern: run the script **every hour**, and let it decide what to do.

Example (on the Barman server):

```
crontab -e
```

Add:

```
0 * * * * /usr/local/bin/barman_backup.sh >> /var/log/barman/barman_backup.log 2
```

This means:

- At **minute 0 of every hour**, cron runs the script
- The script checks day + hour:
- If Friday 00:00 → FULL
- If 08:00 or 16:00 → incremental
- Else → nothing, just log and exit

All output is appended to:

```
/var/log/barman/barman_backup.log
```

So you can monitor:

```
tail -f /var/log/barman/barman_backup.log
```

5. Why This Strategy Works Well

This pattern gives a nice balance:

✅ Weekly Full Backup

- A fresh full backup every week keeps your backup chain **shorter and safer**.
- Restore operations don't need to go through a very long chain of incrementals.

✅ Twice Daily Incremental

- You get more **granular restore points** throughout the week.
- You reduce storage usage vs taking full backups all the time.
- You reduce backup window vs nightly full only.

- The logic is fully visible in one Bash script.
- Easy to explain to auditors, SREs, and teammates.
- Easy to adjust:
- Change full backup day
- Add/remove incremental windows
- Adjust logging paths

6. Possible Improvements

Some ideas you might later add:

Email or Slack alert on failure

1. Check exit code of `/usr/bin/barman backup` and send alert if non-zero.

Different schedule per environment

- Production: full + multiple incrementals
- UAT/Dev: maybe only daily full

Tag backups

1. Use Barman's options (e.g. `--immediate-checkpoint`, `--description`) to label important backups.

Retention policy

1. Combine with `retention_policy` and `retention_policy_mode` in Barman to automatically purge old backups.

7. Conclusion

This small Bash script turns Barman into a **fully automated backup system** with:

- Weekly full backups
- Twice daily incrementals
- Regular maintenance

You get a **simple, predictable, and production-friendly** backup schedule that fits most PostgreSQL environments without adding much complexity.

If you're already running Barman and haven't automated your schedule yet, this is a great pattern to start with — and you can adapt it further to match your RPO/RTO requirements.

Automated Barman Backup + Pgbarman + Barman Incremental Backup + Barman Full Backup +

Written by Sheikh Wasiu Al Hasib

179 followers · 7 following

I am a PostgreSQL DBA with RHCE,RHCSA, Hardening ,CKA and CKS certified. I enjoy to work with database, automation tools like ansible ,terraform, bash etc. .

Follow